Assignment 5: Optimising Attention on a CPU

CS26E003 – Athul Kesav

Single-head attention $O = \text{softmax}(QK^\top/\sqrt{d})\,V$ with $Q, K, V, O \in \mathbb{R}^{N\times d}$, FP32, row-major.

Q1. Machine description

Full details are in `machine_spec.txt`. The CPU is an AMD Ryzen 5 3550H (Zen+): 4 cores and 8 hardware threads (no performance/efficiency split), base clock 2.1 GHz, boost 3.7 GHz, about 3.1 GHz all-core under load (assumed). It supports AVX2 (256-bit ISA), but Zen+ has 128-bit FPU pipes and 2 FMA pipes per core, with 16 YMM registers. Caches: L1D 32 KB, L2 512 KB (private), L3 4 MB (shared), 64 B lines. Memory: 16 GB DDR4-2400, dual channel.

$$
\begin{aligned}
\text{FLOP/cycle/core} &= 2 \text{ FMA pipes} \times \tfrac{128}{32} \text{ lanes} \times 2 \text{ FLOP per FMA} = 16,\\
\text{Peak, 1 core} &= 3.7 \text{ GHz} \times 16 = 59.2 \text{ GFLOP/s},\\
\text{Peak, 4 cores} &= 4 \times 3.1 \text{ GHz} \times 16 = 198.4 \text{ GFLOP/s},\\
\text{Bandwidth} &= 2400 \text{ MT/s} \times 8 \text{ B} \times 2 \text{ channels} = 38.4 \text{ GB/s},\\
\text{Ridge point, 1 core} &= 59.2/38.4 = 1.54 \text{ FLOP/B},\\
\text{Ridge point, 4 cores} &= 198.4/38.4 = 5.17 \text{ FLOP/B}.
\end{aligned}
$$

The all-core peak uses the 4 physical cores: the 8 hardware threads share their FMA pipes, so SMT cannot raise the peak. The 1-core peak assumes the 3.7 GHz boost is held, which a laptop under WSL may not do, so the percentages of peak are if anything pessimistic. The 38.4 GB/s figure is theoretical; STREAM bandwidth was not measured.

Q2. Naive baseline

N	d	time (ms)	GFLOP/s	% of 1-core peak
1024	64	174.9	1.53	2.6
2048	64	705.6	1.52	2.6
4096	64	2902.5	1.48	2.5
4096	128	6198.2	1.39	2.3

Table 1: Naive baseline (1 thread). GFLOP/s is $4N^2d$/time, the driver's convention.

Scaling. From $N = 1024$ to 2048 to 4096 ($d = 64$) the time grows by 4.03× and 4.11×, i.e. quadratically in N, as expected from the $4N^2d$ work. Doubling d (64 → 128 at $N = 4096$) multiplies the time by 2.14×, roughly linear in d. The small excess over 2× comes from the score

and PV stages (2.5× and 2.1×), whose K and V working sets double to 2 MB each, while the $O(N^2)$ softmax is unchanged. GFLOP/s is therefore flat (1.4–1.5), only 2.3–2.6% of the single-core peak.

Where the time goes. The three stages were timed with instrumented copies of the naive code (kept outside `src/`); averages per call:

N	d	scores $QK^\top$	softmax	PV
1024	64	65.0 (38%)	5.4 (3%)	101.4 (59%)
2048	64	275.6 (39%)	24.5 (4%)	399.4 (57%)
4096	64	1100.0 (38%)	94.6 (3%)	1697.0 (59%)
4096	128	2754.7 (43%)	94.0 (1%)	3567.2 (56%)

Table 2: Naive: time per stage in ms per call (share of total).

- *Scores and PV (about 97% of the time)* are dot-product reductions, `acc += a*b`, into a single scalar. Without `-ffast-math` the compiler may not reassociate floating-point additions, so it can neither spread the sum over SIMD lanes nor over several accumulators. Each iteration waits for the previous one: a dependent FMA chain with a latency of about 5 cycles performs 2 FLOP per 5 cycles, i.e. $2 \times 3.7/5 \approx 1.5$ GFLOP/s, which is what is measured. The core is latency-bound, using about 1 of 16 FLOP/cycle.
- *PV is the slower stage (59%).* Its inner loop runs over j and reads `V[j*d+k]` with a stride of d floats (256 B at $d = 64$). Every load touches a different cache line, so besides the dependent chain it has poor spatial locality.
- *Softmax is only 1–4%.* It is $O(N^2)$ against $O(N^2 d)$ for the matrix products, and each 16 KB row stays in L1 across its three passes.
- *Memory traffic is not the problem:* S is 64 MB at $N = 4096$, but moving it a few times at 38 GB/s costs about 10 ms of the 2900 ms.

Q3. Optimisations

The maximum relative error reported by `make check` (limit 10^{-4}) is 0 for loops, tiled and fused at $d = 64$ (they sum in the same order as naive), 1.8×10^{-6} for loops at $d = 128$, and at most 6.1×10^{-6} for best and omp (vector `exp` and deferred normalisation). Against a float64 reference on three rows, `best` has an error of about 6×10^{-8}. The plots `plots/time_N<N>_d<d>.png` show the time of every variant for each input; two of them are shown in Figure 1 (the last bar is `omp` with 8 threads).

(a) loops: reordered loops

What changed. K is transposed once ($K_t[k][j]$). The scores use loop order i, k, j: `s[j] += q*Kt[k][j]` is an element-wise update over contiguous j with no reduction, so clang vectorises it with AVX2 and FMA. PV uses order i, j, k: `o[k] += S[i][j]*V[j][k]` is contiguous in k, and V is read row by row instead of with stride d. The $1/\sqrt{d}$ scale is folded into q (exact for $d = 64$).

What limits it now. Stage times (average per call):

variant	N	d	time (ms)	GFLOP/s	% of peak	speedup	`make check`
`naive`	1024	64	174.9	1.53	2.6	1.0×	reference
`naive`	2048	64	705.6	1.52	2.6	1.0×	reference
`naive`	4096	64	2902.5	1.48	2.5	1.0×	reference
`naive`	4096	128	6198.2	1.39	2.3	1.0×	reference
`loops`	1024	64	22.9	11.71	19.8	7.6×	pass
`loops`	2048	64	89.5	12.00	20.3	7.9×	pass
`loops`	4096	64	410.0	10.47	17.7	7.1×	pass
`loops`	4096	128	689.3	12.46	21.0	9.0×	pass
`tiled`	1024	64	12.0	22.34	37.7	14.6×	pass
`tiled`	2048	64	46.4	23.13	39.1	15.2×	pass
`tiled`	4096	64	224.5	19.14	32.3	12.9×	pass
`tiled`	4096	128	323.6	26.54	44.8	19.2×	pass
`fused`	1024	64	11.2	23.95	40.5	15.6×	pass
`fused`	2048	64	45.4	23.66	40.0	15.5×	pass
`fused`	4096	64	184.2	23.31	39.4	15.8×	pass
`fused`	4096	128	307.6	27.93	47.2	20.2×	pass
`best`	1024	64	6.8	39.71	67.1	25.9×	pass
`best`	2048	64	26.6	40.30	68.1	26.5×	pass
`best`	4096	64	112.5	38.18	64.5	25.8×	pass
`best`	4096	128	232.7	36.91	62.3	26.6×	pass

Table 3: All single-threaded variants. Speedup is relative to naive; % of peak is relative to 59.2 GFLOP/s.

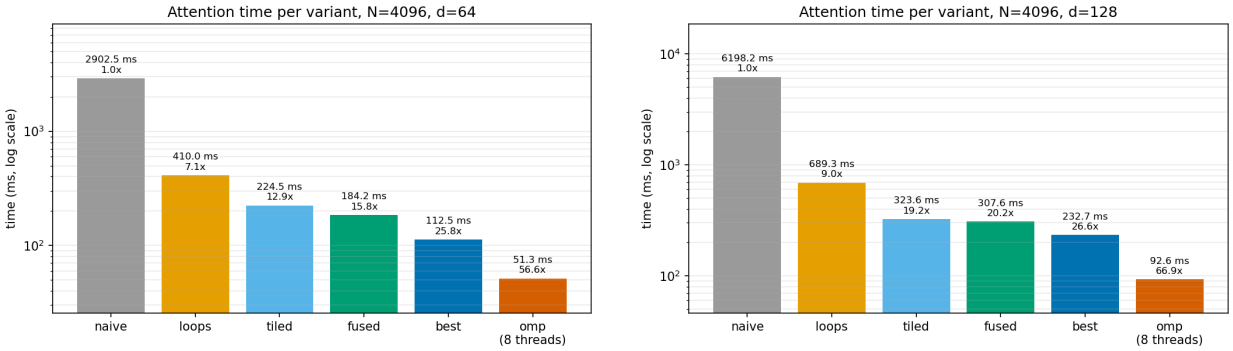


Figure 1: Time of every variant at $N = 4096$, $d = 64$ (left) and $d = 128$ (right).

N	d	scores $QK^\top$	softmax	PV
1024	64	10.1 (41%)	6.3 (25%)	8.2 (33%)
2048	64	40.9 (38%)	27.9 (26%)	38.8 (36%)
4096	64	180.9 (41%)	113.8 (26%)	141.8 (32%)
4096	128	366.6 (47%)	92.8 (12%)	314.3 (41%)

Table 4: Loops: time per stage in ms per call (share of total).

The matrix stages became 6–12× faster (for example 1100 → 181 ms for the scores and 1697 → 142 ms for PV at 4096/64), but softmax did not (scalar `expf`, serial max and sum), so it is now 25–26% of the time at $d = 64$. The two matrix loops are also not FMA-bound: for every FMA they load and store the accumulator row again (`s[j]`, `o[k]`), so they are limited by the L1 load/store ports (2 loads and 1 store per FMA) and reach 10–12 GFLOP/s overall (18–21% of peak). The $N \times N$ matrix is still written to and read from DRAM.

(b) tiled: cache and register tiling

What changed. K and V are packed into contiguous panels of 16 keys and 16 columns, so the micro-kernel streams unit-stride memory (this also avoids the power-of-two stride conflicts of a transposed matrix). Scores and PV are computed by a 4×16 AVX2 micro-kernel: 8 YMM accumulators stay in registers over the whole reduction (k for the scores, j for PV), each loaded B vector is reused by 4 rows and each broadcast A value by 2 vectors, so each step of the reduction needs 6 loads (2 vectors and 4 broadcasts) for 8 FMAs instead of 3 memory operations per FMA. For cache tiling, a block of 32 query rows ($\leq$ 16 KB) stays in L1 while all K panels stream past it; for PV, 16 rows of S are reused against each V panel from L2. Softmax is unchanged.

What limits it now. Time is split between the micro-kernels and the unchanged scalar softmax (about 95–115 ms at $N = 4096$, from the naive and loops stage timings), which is now roughly half of the 224 ms at $d = 64$. This is an estimate, not a separate measurement. It also explains why $d = 128$, with more useful work per softmax element, gains more (26.5 against 19.1 GFLOP/s). S is still a 64 MB matrix that is written once and read twice.

(c) fused: the N-by-N matrix is never stored

What changed. Sixteen query rows are processed at a time. Their scores go into a $16 \times N$ buffer (256 KB at $N = 4096$, which fits the 512 KB L2), softmax runs on those rows while they are cached, and PV consumes them immediately. Only 16 rows of S ever exist, so the DRAM traffic for S disappears (the arithmetic intensity rises from 12.6 to 1024 FLOP/B). The gain over tiled is 224 → 184 ms at 4096/64, and smaller at small N, where S already fit in L3.

What limits it now. Memory is no longer the issue. The scalar softmax (one `expf` and one division per element, $N^2 = 16.8$M of each) is now the largest single cost.

(d) best: fused plus AVX2 softmax

What changed. The fused structure and micro-kernels are kept, with three additions. (1) A hand-written AVX2/FMA `exp` (Cephes polynomial: range reduction $x = n \ln 2 + r$, a degree-7 polynomial for e^r, and 2^n built with integer shifts) processing 16 elements per iteration, with vector max and vector sum, so the whole softmax is SIMD. (2) Deferred normalisation: the division by the row sum is applied to the d outputs of a row (d multiplies) instead of to N probabilities (N divisions). (3) Tuning of the row block R_B: $R_B = 8$ and 16 are within noise of each other (about 115–125 ms at 4096/64), but $R_B = 32$ was 2× slower (230 ms), because the $32 \times 4096 \times 4$ B = 512 KB score block fills the whole L2 and evicts the K/V panels. $R_B = 16$ was kept.

What limits it now. 38 GFLOP/s is about 64% of the nominal single-core peak (and the GFLOP/s metric ignores the roughly 20 FLOP per element spent in `exp`). What remains is the exp/softmax passes (three passes over a 256 KB block that lives in L2, not L1), the broadcast loads in the micro-kernel (1 load per 2 FMAs; Zen+ splits each 256-bit FMA into two 128-bit pipe operations), the write and normalisation of O, the per-call packing of K and V, and possibly a core that does not hold 3.7 GHz under WSL.

Q4. OpenMP

`attn_omp.c` is `best` parallelised with a single `#pragma omp parallel` region. Packing of K and V is shared work (`omp for`), and the loop over 16-row query blocks is an `omp for schedule(static)`. Blocks are independent (each writes only its own rows of O) and each thread owns a private $16 \times N$ score buffer, so there are no locks or atomics on the hot path. Speedup is relative to `omp` with 1 thread, and efficiency is speedup divided by the number of threads.

N	d	threads	time (ms)	speedup	efficiency	GFLOP/s
1024	64	1	6.5	1.00×	100%	41.0
1024	64	2	3.7	1.74×	87%	71.6
1024	64	3	2.5	2.61×	87%	107.0
1024	64	4	1.9	3.37×	84%	138.3
1024	64	6	2.7	2.40×	40%	98.5
1024	64	8	2.3	2.90×	36%	119.0
2048	64	1	26.7	1.00×	100%	40.1
2048	64	2	16.6	1.61×	81%	64.7
2048	64	3	12.5	2.14×	71%	85.7
2048	64	4	12.8	2.10×	52%	84.2
2048	64	6	12.5	2.14×	36%	86.0
2048	64	8	11.9	2.24×	28%	90.1
4096	64	1	111.8	1.00×	100%	38.4
4096	64	2	70.6	1.58×	79%	60.8
4096	64	3	67.7	1.65×	55%	63.5
4096	64	4	67.8	1.65×	41%	63.4
4096	64	6	56.9	1.97×	33%	75.5
4096	64	8	51.3	2.18×	27%	83.7
4096	128	1	235.1	1.00×	100%	36.5
4096	128	2	140.9	1.67×	83%	61.0
4096	128	3	130.9	1.80×	60%	65.6
4096	128	4	127.7	1.84×	46%	67.2
4096	128	6	98.6	2.38×	40%	87.1
4096	128	8	92.6	2.54×	32%	92.7

Table 5: OpenMP scaling of `omp`.

N	d	`best` (ms)	`omp`, 8 thr (ms)	speedup	GFLOP/s	% of 4-core peak
1024	64	6.8	2.3	3.00×	119.0	60
2048	64	26.6	11.9	2.23×	90.1	45
4096	64	112.5	51.3	2.19×	83.7	42
4096	128	232.7	92.6	2.51×	92.7	47

Table 6: `omp` with 8 threads compared with the single-threaded `best`.

Explanation of the scaling.

- *Small N (1024) scales well up to 4 threads* (3.4×, 84% efficiency): there are only 64 row blocks, the whole problem fits in L3, and 4 threads presumably land on 4 distinct physical

cores (threads are not pinned, so the placement cannot be confirmed). With 6 and 8 threads it gets *worse* than 4 (2.4× and 2.9×, although the 8-thread value is noisy): a run lasts only 2–3 ms, so thread start-up, barriers and load imbalance (64 blocks over 6 or 8 threads) are a visible fraction, and the extra threads are SMT siblings of busy cores.

- *Large N (4096) scales poorly:* only 1.6–1.7× with 2 threads and 1.65–1.84× with 4 (efficiency 41–46%), then 2.2–2.5× with 8. Efficiency falls from 79–83% at 2 threads to 27–32% at 8. A likely explanation, not verified with hardware counters, is that every thread streams the whole packed K and V (2 MB at $d = 64$, 4 MB at $d = 128$) from the shared 4 MB L3 for each of its row blocks, in addition to its own 256 KB score buffer, so the combined working set exceeds the L3 and the threads compete for L3 and DRAM bandwidth.
- *SMT and clocks.* Going from 4 to 8 threads adds at most 30–40%, not 2×: the 8 hardware threads share 4 sets of FMA pipes, and the single-thread version already keeps them about 60% busy. All-core clocks (about 3.1 GHz, or lower under the laptop's power limit) are also below the single-core boost.
- Overall, `omp` with 8 threads reaches 84–93 GFLOP/s at $N = 4096$, i.e. 42–47% of the 198.4 GFLOP/s four-core peak, a 56× ($d = 64$) and 67× ($d = 128$) speedup over the naive baseline.

Q5. Roofline

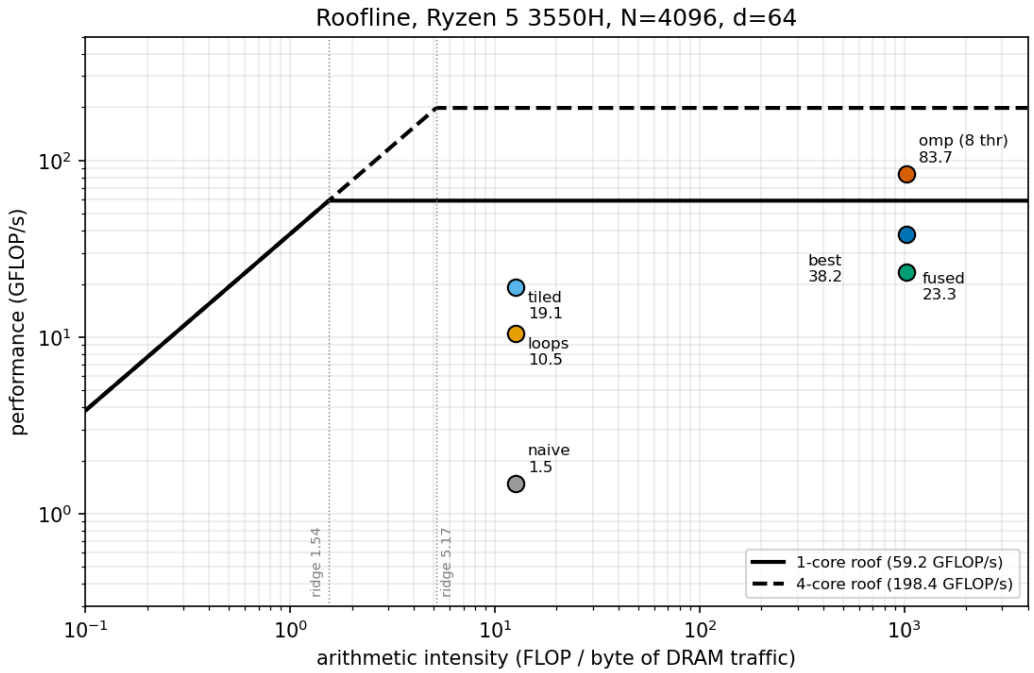


Figure 2: Roofline of the Ryzen 5 3550H with all variants at $N = 4096$, $d = 64$.

The plot (`plots/roofline.png`) uses the Q1 numbers: memory roof 38.4 GB/s, compute roofs 59.2 GFLOP/s (1 core, solid) and 198.4 GFLOP/s (4 cores, dashed), ridge points at 1.54 and 5.17 FLOP/B.

Arithmetic intensity. The work is $4N^2d = 4.29$ GFLOP for every variant (the driver's convention). The bytes are a model of *DRAM traffic*, assuming that anything that fits in cache (K, V, packed panels, a $16 \times N$ block) is read from DRAM only once.

- *naive, loops, tiled* materialise S ($N^2 \times 4$ B = 67 MB, larger than the L3). Step 1 writes S (with write-allocate: one read and one write-back, 2 passes); softmax reads S and writes it back (the 16 KB row stays in L1 across its three passes), 2 passes; PV reads S, 1 pass. That is 5 passes = $20N^2$ = 336 MB, plus the compulsory $16Nd$ = 4.2 MB for Q, K, V, O. The intensity is 4.29 G/340 M = 12.6 FLOP/B ($\approx d/5$).
- *fused, best, omp* never store S, so the traffic is only the compulsory reads of Q, K, V and the write of O: $16Nd$ = 4.2 MB, giving $N/4$ = 1024 FLOP/B. This is an upper bound on the intensity: it ignores the re-reads of the packed K/V panels, which are served from L2/L3 rather than DRAM.

variant	AI (FLOP/B)	achieved GFLOP/s	roof at that AI	% of roof
`naive`	12.6	1.5	59.2	2.5%
`loops`	12.6	10.5	59.2	17.7%
`tiled`	12.6	19.1	59.2	32.3%
`fused`	1024.0	23.3	59.2	39.4%
`best`	1024.0	38.2	59.2	64.5%
`omp` (8 thr)	1024.0	83.7	198.4	42.2%

Table 7: Roofline data at $N = 4096$, $d = 64$ (all points lie to the right of the ridge point).

What limits each variant. At 12.6 FLOP/B even the unfused variants lie to the right of the ridge points, so DRAM bandwidth is *not* what bounds them: moving 340 MB at 38.4 GB/s takes about 9 ms, against 224–2900 ms measured. Their gap to the roof comes from the core.

- *naive* (2.5% of the roof): a single dependent scalar FMA chain (latency-bound, about 2 FLOP per 5 cycles) plus strided access to V.
- *loops* (18%): vectorised, but each FMA is accompanied by a load and a store of the accumulator (L1 load/store-port bound), and scalar softmax is a quarter of the time.
- *tiled* (32%): the kernels are register-blocked and no longer port-bound, but the scalar softmax and the 64 MB round trip of S remain.
- *fused* (39%): the traffic for S is gone and the intensity rises by 80× (the point moves far to the right); the remaining limiter is the scalar softmax (`expf` and division).
- *best* (64% of the 1-core roof): only in-core limits remain (FMA pipe throughput, the exp polynomial, the L2-resident score block, the clock), as listed in Q3(d).
- *omp, 8 threads* (42% of the 4-core roof): although its single-thread kernel reaches 64%, the 8 threads are limited by contention for the shared L3, SMT sharing of the FMA pipes and lower all-core clocks, as discussed in Q4.